

Introspection

The introspection module is module ID 3. It measures resources consumed by the DCGM host engine itself. The public command-line report contains total host-engine memory use and CPU utilization.

When introspection is useful

DCGM monitoring has a host cost: field watches are sampled, values are retained, and client requests are processed. Introspection helps distinguish that cost from the applications being monitored.

Common uses include:

- establishing the host-engine baseline before deploying a monitoring policy;
- comparing overhead before and after adding fields, entities, or faster sampling intervals;
- investigating sustained host CPU or memory growth;
- checking whether monitoring settings fit the node's operational budget.

Introspection reports the host engine as a whole. It does not attribute cost to an individual client, module, group, or field watch, and it does not report GPU workload utilization. Use ordinary DCGM fields and profiling metrics for the workload side of that comparison.

Example: compare monitoring overhead

First take a baseline. The initial query can take longer because DCGM needs two process and system CPU samples before it can calculate utilization.

```
$ dcgmi introspect --show --hostengine
```

The report contains two rows:

- **Memory** is the host-engine byte value divided by 1,024 and displayed with the label **KB**.
- **CPU Utilization** is total host-engine CPU use displayed as a percentage.

Next, start the monitoring configuration you want to evaluate. For example, this command reads GPU temperature and power once per second for GPUs 0 and 1:

```
$ dcgmi dmon --entity-id gpu:0,gpu:1 \  
  --field-id 150,155 --delay 1000
```

While that command is running, repeat the introspection query in another terminal after the values have had time to settle:

```
$ dcgmi introspect --show --hostengine
```

Compare several samples rather than treating one CPU percentage as a durable baseline. If overhead is too high, the usual controls are fewer watched fields or entities, a longer update interval, and a shorter retention period where the consuming command or API exposes those settings.

Interpreting results

The introspection module is lazy-loaded by the first request. `Not loaded` in `dcgmi modules --list` is therefore normal before that request. The module's first CPU result can be delayed while the required samples are collected.

An `Error` value in one report row means that DCGM could not obtain that measurement. In the current command implementation, an individual row can say `Error` even when the command exits successfully, so automation should inspect the values as well as the process status.

Memory and CPU values are observations, not configured ceilings. DCGM does not automatically reject a watch because the host engine crosses a resource budget chosen by the operator.

Further reading

- [dcgmi introspect](#) is the exhaustive command reference, including units and failure behavior.
- [dcgmi modules](#) documents module state and lazy loading.
- [dcgmi dmon](#) documents the example field watch and its connection-scoped lifetime.
- [Metadata](#) documents the C/C++ resource-introspection interfaces, and [Modules](#) documents module status.



[Privacy Policy](#) | [Your Privacy Choices](#) | [Terms of Service](#) | [Accessibility](#) | [Corporate Policies](#) | [Product Security](#) | [Contact](#)

Copyright © 2018-2026, NVIDIA Corporation.

Last updated on 2026-09-10.